The handl platform: a composable host for language toolchains

Draft — proposed, not accepted

handl project

2026-10-21

Abstract

A language toolchain is usually a product: the parser, the formatter, the linter, the typechecker, the evaluator, and the language server are all compiled against the same fixed language definition, and adding a construct means editing every one of them. The handl platform replaces the product with a composer. `handl-platform` defines a single object-safe `Plugin` trait — one model that covers libraries, targets, operations, rules, and options — and a `DefaultHost` that is an empty composer: it holds a capability-indexed plugin registry, runs each plugin's `contribute` method to a fixed point, and then dispatches `CapabilityRequest`s without knowing what tool it has become. Rules are typed contributions delivered to `RuleAcceptor`s through a compile-time-checked `contribute::<P>` channel, with order-free loading via deferred contribution. Operations are themselves plugins (`BasicFormatter`, `BasicLinter`, `BasicCompiler`, `BasicBuilder`, `BasicEvaluator`, `OptionsPlugin`); kits are curated manifests that select plugins and pin providers; and the composition resolver turns `requires`/`optional`/`extends`/ `conflicts`/`targets`/`exclusive`/`ast_kinds` declarations into a checked `ResolvedComposition`. Above the kernel, the engines are data-driven machinery — one abstraction crate per operation in the public `engine-*` family, dispatching on `Language` data — and everything a program can do, down to garbage collection and sandboxing, is a property of the loaded composition rather than a flag in a runtime.

> **!** Important
>
> **Status.** This is a working draft — *proposed, not accepted*. It has not had human revision or external review. It is the platform paper: it covers the composition layer above the kernel — hosts, plugins, rules, kits, options, and the engines that bind them — and cites the kernel paper (handl project 2026e) for the arrow calculus, the `Term`/`Value` model, the `Language` schema, and the graded effect story rather than re-deriving them. Every claim about the implementation names the crate, module, or file that carries it; the repository is `handl-lang/host`, and the API excerpts are drawn from `crates/platform/src/`.

1 Introduction

Two couplings make a language toolchain expensive to grow. The first is *language-to-tool*: each new construct is a new obligation on the parser, the evaluator, the typechecker, the formatter, the linter, and every analysis, so tools disagree in proportion to the language's surface. The kernel paper (handl project 2026e) attacks that coupling by making a language pure data — a `Language` value assembled from fragments — and making every engine generic over it.

The second coupling is *tool-to-host*, and it is this paper's subject. Even with a data-driven language, someone must decide which languages, engines, operations, library functions, and options a given running tool contains. The usual answers — a hardcoded main, a bespoke plugin interface per subsystem, an extension registry per tool — reproduce the original problem one level up: every subsystem invents its own composition rules, and two tools that should share components cannot.

The handl platform's answer is to refuse the distinction. Everything a host loads is a `Plugin`; the only real sub-distinction is whether a plugin also implements `RuleAcceptor` — whether it accepts contributions. A library is a plugin that registers more plugins. A target is a plugin that contributes rules for a compilation-shaped operation. An operation is a plugin that accepts rules and answers a capability. An option set is a plugin that accepts schemas. A formatter, an evaluator, a garbage collector, and a module of host functions are the same kind of thing viewed from the composer.

Concretely, the paper describes:

- Section 3 — the composable host: `Plugin`, `CapabilityId`, `CapabilityRequest`/`CapabilityResponse`, `PluginRegistry`, and `DefaultHost`'s `load`/`run` lifecycle;
- Section 4 — rules as typed contributions: `Contribution`, `RuleAcceptor`, `Operation`, `Behavior`, and the `contribute::<P>` channel with deferred delivery;
- Section 5 — operations as acceptor plugins, including the `Eval` operation whose context *is* a serializable continuation;
- Section 6 — manifests and composition resolution: `PluginManifest`, `resolve`, and the `ResolvedComposition`;
- Section 7 — kits and configuration: `Kit`/`KitManifest`, `HostConfig`/`.handl.toml`, and `OptionsPlugin`/`ResolvedOptions`;
- Section 8 — the engines as data-driven machinery in the `engine-*` family, bound by name through `handl-runtime`'s `EngineRegistry`;
- Section 9 — `host-tensor`, a worked example of a capability surface composed entirely through the plugin model;
- Section 10 — the design principles the model operationalises;
- Section 11 — the public workspace map and the honest boundaries.

The design claim is narrow: *one plugin model plus a capability-indexed registry plus a fixed-point contribution pass is sufficient to assemble every tool in a language toolchain from the same pieces.* The evidence is the repository itself: the composer is roughly two hundred lines of `host.rs`, the resolver is data over manifests, and nothing in `handl-platform` knows what a "language" is.

2 The layer below

The platform sits on the kernel, and this section is deliberately short — the kernel paper owns the calculus narrative (handl project 2026e). What this layer needs to know:

- Everything that executes is a `Term` in a closed six-combinator arrow calculus (`arr`, `>>>`, `first`, `choice`, `loop`, `app`), extended with `Op`/`Handle` for graded algebraic effects; arrow values are self-contained serializable data, not closures (Hughes 2000; Lindley, Wadler, and Yallop 2010; Sanada 2024; handl project 2026e).
- A language is a `Language` value — a set of fragment tables (`tokens`, `lexer_rules`, `grammar`, `ast`, `eval`, `types`, `transforms`, `options`, `laws`, `rewrites`, …) merged with override/collision semantics, not code compiled into an engine.
- Engines are *models* of that data: the staged contract is `lex(lang, src)`, `parse(lang, toks)`,

`eval(lang, ast)`, and `run = eval ∘ parse ∘ lex`; dispatch on node kind goes through `lang.eval`/`lang.grammar`/`lang.ast` child roles, never a hardcoded `match`.

The platform's job is everything the kernel refuses to do: decide *which* `Language` is loaded, *which* engines interpret it, *which* operations exist, *which* host functions programs may call, and *what configuration* applies. Where the kernel says "a model interprets terms", the platform says "a host loads the plugins that provide the models". The kernel's `ArrSource::Foreign` and `ModuleLookup`/`Function` seam is exactly where the platform's world of host functions plugs in (Section 9).

3 The composable host

3.1 One trait

`handl-platform` (`crates/platform`) defines the universal, object-safe extension point. The real signature — verified against `crates/platform/src/plugin.rs` — is:

```
pub trait Plugin: Send + Sync + Any {
    fn id(&self) -> &str { self.name() }
    fn name(&self) -> &str;
    fn version(&self) -> &str { "0.0.0" }
    fn capabilities(&self) -> &[CapabilityId];
    fn requires(&self) -> &[CapabilityId] { &[] }
    fn optional(&self) -> &[CapabilityId] { &[] }
    fn metadata(&self, capability: &CapabilityId) -> Option<CapabilityMetadata>;
    fn manifest(&self) -> Option<&PluginManifest> { None }

    fn contribute(&self, _ctx: &mut ContributionContext)
        -> Result<(), PlatformError> { Ok(()) }

    fn run(&self, request: &CapabilityRequest, ctx: &mut PluginContext)
        -> Result<CapabilityResponse, PlatformError>;

    fn module_lookup(&self) -> Option<Rc<dyn ModuleLookup>> { None }
}
```

Three phases are visible in the signature. **Discovery** is declarative: `id`/`name`/`version`, the `capabilities` a plugin provides, the `requires`/`optional` capability dependencies it declares, and an optional `manifest` for plugins that participate in composition resolution (Section 6). **Composition** is `contribute`: during `load`, each plugin gets a mutable `ContributionContext` through which it can register more plugins and deliver typed rules to acceptors (Section 4). **Execution** is `run`: a capability request plus a `PluginContext` that hands the plugin mutable access back to both registries.

The capability wire format is deliberately thin:

```
pub struct CapabilityRequest {
    pub capability: CapabilityId,
    pub payload: serde_json::Value,
}
pub struct CapabilityResponse(pub serde_json::Value);
```

`CapabilityId` is a newtype string; `CapabilityMetadata` is `Value`. A capability request is serializable end to end — which is what lets the same dispatch serve in-process closures, manifest-loaded plugins, and out-of-process providers (Section 7).

3.2 The registry and the host

`PluginRegistry` is a vector of `Arc<dyn Plugin>` plus a `HashMap<CapabilityId, usize>` index (`plugin.rs`). `register` indexes each declared capability; `resolve(&cap)` returns the provider; `run` resolves and invokes; `get_plugin_mut::<P>`/`get_plugin::<P>` downcast to concrete types — the channel `contribute::<P>` is built on. Two registries is all the state a host carries:

```
pub struct DefaultHost {
    plugins: PluginRegistry,
    kits: KitRegistry,
}
```

`DefaultHost` is an *empty composer*. Its lifecycle is two calls:

- **`load(&mut self) -> Result<(), PlatformError>`** — the contribution pass. `host.rs` iterates the registry, calling each plugin's `contribute` exactly once in registration order; because plugins registered during the pass extend `plugins.len()`, the loop bound grows until no new plugins appear — a fixed point over registration order. After each `contribute`, deferred rule contributions are retried (Section 4.2); after the pass, pending exclusions are applied and any still-unresolved acceptor names fail with `MissingRuleAcceptor`.
- **`run(&mut self, &CapabilityRequest) -> Result<CapabilityResponse, PlatformError>`** — capability dispatch: resolve the capability to its provider, invoke `plugin.run` with a `PluginContext` borrowing both registries. An unprovided capability is `PlatformError::UnknownCapability` not a panic.

`resolve_composition` is the read-side validation: it builds manifests from the live registry (`Plugin::manifest`, synthesised when absent) and runs the manifest resolver (Section 6) against them, so a host can ask "is this composition coherent?" without a kit.

```
let mut host = DefaultHost::new();
host.register_plugin(GreetPlugin::new());   // provides "greet"
host.load()?;
let resp = host.run(&CapabilityRequest {
    capability: "greet".into(),
    payload: json!({"name": "alice"}),
})?;
```

The example is the published tutorial's (`docs/site/tutorials/build-a-host.md`, and `write-a-plugin.md` adds the contribution path); it is included because it is the whole idea — the host started empty and ended as a tool that can `greet`. Nothing distinguishes this from a language server except the plugins loaded.

Two stock plugin forms cover the common providers. `InProcessPlugin` wraps a closure `Fn(&CapabilityRequest, &mut PluginContext) -> Result<CapabilityResponse, PlatformError>` as a plugin. `ProcessPlugin` represents an out-of-process provider: with no run function it reserves the capability id and answers `NotImplemented`; `with_run_fn` makes it live.

`CallableLookup` (a two-getter trait over the registries) and `Vm<CallableLookup, Env>` (`vm.rs`) de-

couple the minimal VM wrapper from `DefaultHost` itself — the "VM" is a host plus an environment, generic over both.

4 Rules as typed contributions

The contribution vocabulary is four traits (`crates/platform/src/plugin_model.rs`):

```
pub trait Contribution: Send + Sync + 'static {
    fn id(&self) -> &str;
}

pub trait RuleAcceptor: Plugin {
    type Rule: Contribution;
    fn accept(&mut self, rule: Self::Rule) -> Result<(), PlatformError>;
}

pub trait Operation: Send + Sync + 'static {
    type Request;
}

pub trait Behavior<Op: Operation>: Send + Sync + 'static {
    fn kind(&self) -> &str;
    fn apply(&self, op: &mut Op::Request) -> Result<(), PlatformError>;
}
```

An `Operation` is a marker type whose associated `Request` is the mutable record a rule acts on. A `Behavior<Op>` handles one `AstNode.kind` — and the AST itself is not an enum but a single open node type:

```
pub struct AstNode {
    pub kind: String,      // keyed to registered Behaviors
    pub span: Option<Span>,
    pub payload: serde_json::Value,
}
```

So "a formatter rule for `comment` nodes" is `impl Behavior<Format>` with `kind() == "comment"`, and "a new language construct" is a set of behaviors — one per relevant operation — contributed for a kind the platform has never heard of. This is the platform-level counterpart of the kernel's dispatch-on-data rule: the platform does not know the operations' node vocabularies either.

4.1 `contribute::<P>`: typed delivery

`ContributionContext` is what a plugin sees during `load`. Its key method:

```
pub fn contribute<P: RuleAcceptor>(&mut self, rule: P::Rule)
    -> Result<(), PlatformError>
```

The type parameter `P` pins the acceptor's `Rule` associated type at compile time: a `Format` behavior cannot be contributed to a linter — the delivery is type-safe, while the *addressing* is dynamic (by concrete type, resolved through `get_plugin_mut::<P>`). The context also offers `register_plugin`, `resolve`/`run` for invoking capabilities of already-loaded plugins during composition, `has_plugin_id`,

`exclude_plugin` (deferred removal), and `contribute_exec_control` for the evaluator's hook channel (Section 5).

4.2 Deferred contribution

`contribute::<P>` does not require `P` to be loaded first. If the acceptor is absent, the rule is boxed into a `PendingRule` — a closure over the rule slot that retries `accept` against the registry — and queued. `DefaultHost::load` calls `flush_pending_rules` after every `contribute`, so a rule is applied as soon as its acceptor appears; any acceptor names still unresolved at the end fail the load with `MissingRuleAcceptor`. Loading order is therefore a convenience, not a constraint: a kit may register a rule plugin before the operation it targets, and a library plugin's `contribute` may register the acceptor itself.

5 Operations as acceptor plugins

The platform ships the generic operation contracts in the same crate. Each follows one shape — a marker `Operation`, a `Request` record carrying the `AstNode` plus `ResolvedOptions`, a `Behavior<Op>` alias, and a `Basic*` `RuleAcceptor` plugin that holds `rules` keyed by kind:

|| Operation | Request fields | `Basic*` acceptor | Capability | ||—|—|—|—| || `Format` | `ast`, `output`, `options` | `BasicFormatter` | `fmt` | || `Lint` | `ast`, `diagnostics`, `options` | `BasicLinter` | `lint` | || `Compile` | `ast`, `compiled`, `options` | `BasicCompiler` | `compile` | || `Build` | `ast`, `built`, `options` | `BasicBuilder` | `build` | || `Eval` | `ast`, `ctx`, `options`, `result` | `BasicEvaluator` | `eval` |

: The built-in operation acceptors (`format.rs`, `lint.rs`, `compile.rs`, `build.rs`, `eval.rs`). {#tbl-operations}

Dispatch is uniform: `run` lifts the request payload into an `AstNode` whose `kind` selects which registered rules `apply`. These `Basic*` acceptors are the *contribution seam* — the place per-kind rules land — not the canonical toolchain engines; the actual pipelines are the data-driven `engine-*` layer bound over `Language` (Section 8). The two exist for different reasons and the crate documents the seam honestly: acceptors compose Rust behavior per node kind; engines interpret `Language` data wholesale.

5.1 Eval is the rich operation

`Eval` is where the uniform pattern needs more, and `eval.rs` shows why the model still holds. `EvalRequest` carries an `EvalContext`: a stack of variable scopes, `EvalLimits` (`max_depth`, `max_calls`), a call counter, and — critically — an `EvaluatorCallback` recursion handle so a rule evaluates children via `req.ctx.eval(&child)`.

Two consequences matter:

- **The context is the continuation.** `EvalError::Yield { context }` suspends evaluation; the context serializes (runtime handles are `#[serde(skip)]`), and `BasicEvaluator::resume` re-attaches the evaluator and module lookup and continues. A `Host` can catch `PlatformError::Yield`, persist the context, and resume later — the resumable-computation story of the kernel's `Op`/`Handle` carried into the operation layer.
- **Cross-cutting execution is a two-hook seam, not middleware.** `ExecControl` contributes `before_eval`/`after_eval` hooks invoked around every dispatch; `before_eval` may return `ControlFlow::Skip(value)` to short-circuit with a cached result. Limits, tracing, caching,

and sandboxing are contributed rules — delivered through `contribute_exec_control` with the same deferred machinery — rather than evaluator internals.

The module boundary is the last piece of the eval story. `Module`/`Function`/`ModuleLookup` define qualified host functions (`math.add`), `ModuleRegistry` (in `plugin-module`) resolves them, and any plugin may expose a registry through `Plugin::module_lookup`. The evaluator wires the first registry it finds into `EvalContext`, so `call_function("math.add", …)` works for every contributed rule without the evaluator knowing any module names. This is "the host provides the world" (Section 10) made concrete: which functions exist, which namespaces are available, and what effects are permitted are all load-time composition facts.

6 Manifests and composition resolution

Discovery in the small is `capabilities()`/`requires()`/`optional()` on the trait. Discovery in the large — before anything is registered — is `PluginManifest` (`manifest.rs`), pure serializable data:

|| Field | Type | Meaning | ||—|—|—| || `id`, `version` | `String` | identity; version defaults `"0.0.0"` | || `provides` | `Vec<CapabilityId>` | capabilities offered | || `requires` / `optional` | `Vec<Dependency>` | hard / soft deps — `Dependency::Capability{capability, version}` or `Dependency::Plugin{id, version}` | || `extends` | `Vec<ExtendsDeclaration>` | plugin → extendee (+ optional `extension_point`) | || `conflicts` | `Vec<ConflictDeclaration>` | plugin- or capability-level exclusions, with `reason` | || `replaces` | `Vec<String>` | plugin ids superseded | || `targets` | `Vec<TargetCompatibility>` | per-target `TargetSupportLevel`: `Supported`/`Exact`/`Lossless`/`Lossy`/`Checked`/`Emulated`/`Unsupported` | || `exclusive` | `Vec<CapabilityId>` | capabilities that may not have multiple providers | || `metadata` | `Value` | opaque loader data (e.g. a native library `path`) | || `ast_kinds` | `Vec<AstKindDeclaration>` | `Produces`/`Consumes` claims over node kinds |

: The `PluginManifest` schema. {#tbl-manifest}

`resolve(manifests, selectors, target)` (`resolve.rs`) is the composition checker — a deterministic pass over the manifest data, not a SAT solver:

1. Reject two manifests with the same id at different versions.
2. Build the provider map: a `CapabilitySelector` pins a provider (validated — the pin must actually provide the capability); a capability with one provider maps to it; multiple providers are an error only if the capability is `exclusive`, otherwise the highest semver wins.
3. Check every `requires`: capability deps resolve through the provider map, plugin deps by id, both with `VersionReq` checking — failures are `MissingCapability`/`MissingPlugin`/`VersionMismatch`. Unresolved `optional` deps are *recorded* in `missing_optional`, not failed.
4. Check `conflicts`; build `extension_wirings` from `extends`.
5. Emit a deterministic `initialization_order` — Kahn's algorithm over requires+extends edges with a `BinaryHeap` for stable output; an unschedulable graph reports the actual `CyclicGraph` cycle.
6. Produce a per-plugin `target_report` against the requested target, and validate `ast_kinds`: a consumed kind that some available plugin could produce must have a selected producer (`MissingProducer`).

The result is `ResolvedComposition` — selected plugin ids, the capability→provider map, extension wirings, init order, missing optionals, target report. `ResolutionError` is exhaustive and named per

failure mode. `DefaultHost::resolve_composition` runs the same check over a *live* registry via `iter_manifests`, so the manifest world and the registry world meet at one function.

7 Kits and configuration

7.1 Kits are curated manifests

A `Kit` is the unit of reuse: a `KitManifest` plus the `available_plugins` the kit crate ships (`kit.rs`).

```
pub struct KitManifest {
    pub id: String,
    pub name: String,
    pub plugins: Vec<String>,              // selected plugin ids
    pub target: Option<String>,            // optional target runtime
    pub selectors: Vec<CapabilitySelector>,// provider pins
}
```

`Kit::resolve` filters `available_plugins` to the manifest's selection and runs the same `resolve` (Section 6) with the kit's selectors and target — curation is manifest-level, so a kit is a *closed* list: the host does not discover plugins on disk, and a kit requesting an absent plugin fails loudly. A kit manifest loads from TOML or JSON (`KitManifest::load`). Kits may also be assembled from authored fragment directories — the language-as-data path published by `handl-kit-prelude-lang` (`language_from_dir`), where a "kit" is a directory of YAML `tokens/grammar/eval/types/ast/caps/ options/lexer_rules` files merged into a `Language` (handl project 2026c); external producers can emit the same schema — the research stack (`graphopt`) already does (mlopt project 2026).

7.2 `.handl.toml` and `HostConfig`

`HostConfig` (`config.rs`) is the in-memory shape of `.handl.toml`: kit references with per-kit `exclude`, direct `plugins`, global `excludes`, option `overrides`, and `dependencies`/`patch` tables whose `DependencySpec` (path or git+rev/branch/tag, with `subdir`) resolves package sources — including the dev-link story where a `[patch]` entry redirects a committed git source to a sibling checkout. `HostConfig::from_toml` parses it; the applier crates (`plugin-applier*`) turn it into registrations and overrides on a `Host`. Excluding a kit's plugin does not fork the kit — and it is configuration, not a security boundary: a loaded plugin shares the process.

7.3 Options are contributions too

`OptionsPlugin` (`options.rs`) is a `RuleAcceptor` whose rule type is `OptionSchema` — the option system uses the same contribution channel as every other rule kind:

```
pub struct OptionSchema {
    pub id: String,
    pub scope: OptionScope,          // Global | Kit | Node
    pub applies_to: Option<String>,
    pub default: Value,
    pub possible_values: Vec<Value>,
    pub description: String,
}
```

A plugin declares its own tunables with `contribute::<OptionsPlugin>`; `OptionsPlugin::resolve` merges schema defaults with overrides delivered through the `option/set` capability into a `ResolvedOptions` (`get`/`choice`/`is_enabled`), which every operation `Request` carries. Because options live in the contribution graph, a plugin can introduce and consume a tunable without touching the host or any other plugin — the formatter does not know `comment.style` exists; it reads the resolved value from its request.

7.4 Loading is itself a plugin

The bootstrapping is complete: plugin *loading* is an acceptor with contributed rules. `plugin-load` defines `PluginLoader` (`can_load`/`load` over a `PluginManifest`), a `LoaderRule` contribution, and a `LoaderHostPlugin` exposing a `load` capability. `plugin-loader-native`'s `NativeLoader` resolves `manifest.metadata.path` to a shared library, binds the `handl_plugin_run` C symbol, and wraps it in a `Plugin` whose `run` serializes the `CapabilityRequest` to JSON, crosses the boundary, and deserializes the response — with `check_portable` rejecting non-serializable payload values. The extension ABI itself (JSON over a C boundary) is kernel (handl project 2026e); who loads what is platform policy, and here the policy is: the loader is a plugin that contributes a loader rule to a loader host.

8 Engines as data-driven machinery

The `engine-*` layer is the platform's second half: for each stage of the language pipeline there is an **abstraction crate** — the operation marker, the request type, the engine trait — and a **concrete engine** that interprets `Language` data. All are public repos in the same family, and all obey the kernel's rule: dispatch on `Language` data, never a hardcoded kind match.

|| Crate | Contract | Concrete engines | ||—|—|—| || `hnl-engine-lex` | `Lexer` marker; `LexerRequest{source, options}`; `LexerEngine::lex(&self, lang, &req) -> Result<Tokens, LangError>` | `hnl-engine-lex-pattern` — tokenizes via `LexerRuleDef` fragments | || `hnl-engine-parse` | `Parser` marker; `ParserRequest{tokens, options}`; `ParserEngine::{expression(ctx, rbp), parse(lang, &req)}`; `ParserContext` token cursor | `hnl-engine-parse-pratt` — a Pratt engine over `lang.grammar` | || `hnl-engine-transform` | `Transform` marker; `TransformRequest`/`TransformMode`/`Transfor` `TransformEngine::{transform, find_matches}`; `RuleSource`, the `NativeRewrite` seam | rewrite engines interpret `lang.transforms` | || `hnl-engine-exec` | `Eval` marker; `EvalRequest{ast, options}`; `EvalEngine::eval(&self, lang, &req) -> Result<Value, LangError>` + `with_modules`/`with_glo` wiring | `hnl-engine-exec-tree-walk` — the reference interpreter |

: The staged engine contracts. {#tbl-engines}

`hnl-engine-exec-tree-walk` deserves one sentence: it is the reference `Model` — it looks up `lang.eval[node.kind]`, obtains the `EvalSchemeDef`'s `Term`, and walks the term graph directly, including the arrow-value record form (`{"__arrow__": true, …}`), the pair/either payload conventions, causal `Loop` semantics, and the no-scheme fallback that returns `node.payload`. It carries zero Rust knowledge of any concrete construct — the same `Term` machinery the kernel paper describes, interpreted (handl project 2026e). The staged arrow-engine program (staging to explicit process graphs, spatial execution) is a separate interpretation strategy on the same terms, kept as a research note (Emidio 2026).

8.1 Binding engines by name

`handl-runtime` (`crates/handl-runtime`) is the composition layer for engines. It names no engine crate:

- `EngineRegistry` is five name→engine maps — lex, parse, eval, typecheck, compile — populated by the host (`register_lex`/`register_parse`/`register_eval`/…).
- `RuntimeConfig` is serializable name-based selection (`{lex: "data", parse: "pratt", eval: "tree-walk", typecheck: ""}` — empty means unbound), so a kit or config file selects engines *as data*; swapping an engine is a config change.
- `Runtime::from_registry` resolves names against the registry and exposes staged `lex`/`parse`/`eval`/`compile`/`run_compiled`; an unbound stage returns `LangError::NotImplemented`, which makes eval-only runtimes (a compiler consuming a pre-parsed `Ast`) a natural configuration rather than a special mode.
- `RuntimeFactory`/`RuntimeHandle` let leaf plugins obtain a sub-runtime bound to a `Language` without reaching up the dependency graph — the factory re-injects itself so modules can recurse.

The eval operation and the eval *engine* are distinct and both real: `BasicEvaluator` (Section 5) composes contributed Rust `EvalBehavior`s per node kind and owns the yield/exec-control story; an `EvalEngine` interprets `lang.eval` schemes wholesale. A host may bind either — they plug into different seams of the same model.

9 host-tensor: a capability surface end to end

`handl-host-tensor` (`handl-lang/host-tensor` (handl project 2026b)) is the public worked example of everything above, in one component. It provides a dense row-major `f64` tensor behind a `tensor.*` operation surface — 28 declared ops (creation, numpy-broadcasting elementwise arithmetic, `matmul`, `conv2d`, pooling, shape ops) plus a `tensor.derive` registry of hand-maintained adjoints — and it enters a host without any platform change:

```
let module = HandlModule::new("tensor", TENSOR_HNL, &[])
    .with_ops(TENSOR_OPS, Arc::new(tensor_op_registry()));
// registered into the runtime's ModuleRegistry;
// a foreign call to `tensor.matmul` reaches the Rust impl.
```

Three boundary decisions make it a clean example rather than a special case:

- **The value crosses as data.** On the `serde_json::Value` boundary a tensor is `{"__tensor__": {"shape": [d,…], "data": [x,…], "dtype": "f64"}}`. Engines that know the kernel's `TaggedValue::Tensor` decode the tag at the host-call boundary and carry it opaquely; engines that do not see a plain object. No engine or language edit was needed to *carry* tensors.
- **Dispatch is the `ModuleLookup` seam.** The crate ships its interface source plus the Rust atoms; a runtime composes it into a `HandlModule`, registration places it in the module chain, and `call_host("tensor.matmul")` resolves through `ModuleLookup` to the `OpFunction` and then the Rust implementation — the same path every module uses.
- **Engine choice stays data.** A `HandlModule` can select its eval engine by name through the same `EngineRegistry` (`eval_name`), keeping the swap a configuration fact.

The result: a host gains a numeric capability surface — including registered adjoints an optimisation loop can dispatch by name — by registering a module, exactly as a host gains `fmt` by registering an

acceptor.

10 Design principles

The platform's published principles (`docs/site/explanation/ design-principles.md` (handl project 2026f)) are the commitments this design operationalises:

1. **Everything is a plugin.** Semantics, tooling, targets, GC, the standard library — one trait. Nothing is privileged except the composition vocabulary itself (`Plugin`/`RuleAcceptor`/`Operation`/ `Behavior`/`Contribution`/`Host`).
2. **The host provides the world.** No built-ins: which functions exist and which effects are permitted is decided by loaded plugins (the `ModuleLookup` seam), in the capability sense of Miller (Miller 2006) and the effects-as-capabilities reading (Brachthäuser, Schuster, and Ostermann 2020) — what the context provides *is* the effect surface.
3. **Evaluation is an operation, not a subsystem.** There is no `VM` alongside the operation system; `Vm` is a generic pairing of a `CallableLookup` and an environment.
4. **A language is data; engines dispatch on it.** The `engine-*` contracts take `&Language`; a renamed-kind language runs untouched.
5. **Kit-level properties are composition facts.** Sandboxing is "which plugins are loaded"; GC is "which plugin provides it"; targets are rules. None is a kernel or platform flag.
6. **Composition over registries.** One registry, one loading algorithm, fixed-point contribution; crate boundaries exist only at real dependency seams.

The same ideas recur as engineering choices: `contribute::<P>` is type-safe delivery with dynamic addressing; deferred contribution makes order a convenience; manifests make composition *checkable data*; `RuntimeConfig` makes engine choice *checkable data*; the loader seam makes even loading *a plugin*.

11 Implementation status

The platform layer is `handl-lang/host`, a cargo workspace; the pieces this paper describes are:

|| Crate (all under `crates/`) | Contents | ||—|—| || `platform` (`handl-platform`) | `Plugin`/`Capability*`/`PluginRegi `Contribution`/`RuleAcceptor`/`Operation`/`Behavior`/`ContributionContext`; the `Format`/`Lint`/`Compile`/`Build`/`E contracts + `Basic*` acceptors; `OptionsPlugin`; `PluginManifest`/`resolve`/`Kit*`/`HostConfig`; `AstNode`; `Boundary`; `Vm` | || `plugin-module` | `ModuleRegistry` (`ModuleLookup` impl), `ModuleRuntime` | || `plugin-load`, `plugin-loader-native` | `PluginLoader`/`LoaderRule`/`LoaderHostPlugin`; the cdylib `NativeLoader` | || `plugin-applier*` | `HostConfig` → host application (`.handl.toml`, CLI, env) | || `plugin-ext-*` | published capability *contract surfaces* for the operations layer | || `plugin-lib-*` | library plugins — module implementations such as `HandlModule`/`OpFunction` (`plugin-lib-handl`) | || `handl-runtime` | `EngineRegistry`, `RuntimeConfig`, `Runtime`, `RuntimeFactory`/`RuntimeHa | || `handl-kit-*` | kit data assembly — merging `Language` fragments and manifests into loadable kits |

: Host workspace map (public crates; `handl-platform` is the shared contract). {#tbl-crates}

What is *proven* by the tree: a binary that depends only on `handl-platform` builds a working tool (`docs/site/tutorials`), the deferred-contribution and resolution paths are unit-tested in the crate, and the full pipeline — a `Language` assembled from `kit-prelude-lang` fragments, interpreted by `engine-*` engines, extended by `host-tensor` modules — is exercised end to end.

Honest boundaries, stated rather than elided:

- **Contribution order is registration order.** `load` runs `contribute` once per plugin in the order registered (with newly-registered plugins visited as the loop grows); it is *not* itself the manifest topological order — `resolve` produces that order for kits, and deferred rules cover acceptor timing. A kit that wants dependency ordering uses the resolver's `initialization_order`.
- **Capability dispatch is last-registration-wins.** The registry index maps a capability to the most recently registered provider; the resolver's `provider_map` (selectors, unique provider, highest semver, `exclusive` checks) is the *manifest-level* answer, and `resolve_composition` surfaces disagreements rather than enforcing them during `register`.
- **The boundary is JSON.** `CapabilityRequest`/`CapabilityResponse` and the `AstNode` payload are `serde_json::Value`; non-finite floats and non-portable values are boundary concerns the native loader checks explicitly (`check_portable`).
- **Not a security model.** Kit exclusion and `exclusive` resolution are correctness tools; plugins share the process.
- **Above the contracts is unpublished.** The command hosts, the release toolchain engines, and the curated kits exist as private siblings; the seams they plug into are the public ones described here.

12 Related work

Plugin architectures. The contribution model descends from the extension-point tradition: Gamma and Beck's Eclipse model, where a plugin declares extension points and contributes extensions to others' (Gamma and Beck 2003), and OSGi's service registry with declarative wiring (OSGi Alliance 2020). The platform's difference is that the extension *points* are typed (`RuleAcceptor::Rule`) and delivery is the compiler-checked `contribute::<P>`; Eclipse's loosely-typed extension registry becomes an associated-type constraint. Fixed-point `contribute` is what those systems call declarative assembly done imperatively — plugins may register plugins mid-load.

Language workbenches and extensible compilers. Spoofax (Kats and Visser 2010), Rascal (Klint, Storm, and Vinju 2009), JastAdd (Ekman and Hedin 2007), Silver (Van Wyk et al. 2010), and Kiama (Sloane 2011) all attack the language tool coupling with declarative rule systems over syntax; the workbench survey of Erdweg et al. (Erdweg et al. 2013) is the landscape. The handl split is different in kind: those systems make *rules* declarative inside a fixed workbench, while here the *language* is data and the *tools* are plugins — the two axes are attacked separately and the composer between them is this paper. The independently-extensible problem is the classical expression problem (Zenger and Odersky 2005); data-driven AST kinds plus per-operation behaviors is the platform's answer — a new construct is new rules for existing operations, and a new operation is a new marker type, neither touching the other.

Polyglot hosts and staged interpreters. Truffle/Graal is the polyglot host precedent — one VM hosting many language implementations (Würthinger et al. 2013), with partial evaluation turning interpreter+program into compiled code (Würthinger et al. 2017). The kernel cites the Futamura projection for the same structure (handl project 2026e); the platform's version is coarser and more modular — engines are exchangeable by name in a registry rather than by partial evaluation of an interpreter, though the staged-arrow research program aims at the same territory (Emidio 2026). Finally-tagless interpretation (Carette, Kiselyov, and Shan 2009) is the typed cousin of "one term, many interpreters" — here the tagless repr is the `Term` IR and the interpreters are registered engines. Language servers externalise tooling behind a protocol (Microsoft 2023); the capability request is

the in-process analogue, with `CapabilityId` playing the method-name role. Lua's embeddability — the host owns the runtime, not vice versa (Ierusalimschy, Figueiredo, and Celes 2007) — is the deployment stance: `handl-platform` is a library you compose, not a VM you embed into.

Within the org. The kernel paper owns the calculus and the `Language` schema (handl project 2026e); the research prototype and the ADR series carry the decision history (handl project 2026a, 2026d); the platform documentation carries the vetted explanations this paper condenses (handl project 2026f). The theory-as-data / model-as-functor reading (Lawvere (Lawvere 1963)) is the same one the kernel claims: a `Language` is the theory, an engine is a model, and — one level up — a `PluginManifest` set is the composition's data and `resolve` its checker.

13 Limitations and open questions

- **Composition is checked, not solved.** `resolve` is a deterministic pass, not a constraint solver: no backtracking version negotiation, no dependency *search*. Version ranges are verified (`VersionReq`) but not solved for. Whether the manifest vocabulary needs a real solver is open.
- **Dispatch is linear over rules.** `Basic*` acceptors scan their rule list per node kind; hot paths live in the engines, and a kind-indexed dispatch table is a plausible optimisation that has not been needed.
- **The two eval paths coexist by design but share no scheduler.** `BasicEvaluator` (behavior rules, yield, exec-control) and `EvalEngine` (scheme interpretation) answer different needs; a unified story — e.g. exec-control hooks inside scheme evaluation — is not yet designed.
- **Deferred contribution makes loads order-tolerant, not order-independent.** A `contribute` that *reads* another plugin's state still observes registration order; only rule delivery is deferred.
- **The contract surface is ahead of the released tools.** The public contracts describe operations whose production engines are private; the honesty cost is that `Basic*` acceptors must not be mistaken for the canonical pipelines — this paper has tried to keep the two roles named distinctly.

Brachthäuser, Jonathan Immanuel, Philipp Schuster, and Klaus Ostermann. 2020. "Effects as Capabilities: Effect Handlers and Lightweight Effect Polymorphism." *Proceedings of the ACM on Programming Languages* 4 (OOPSLA). https://doi.org/10.1145/3428194.

Carette, Jacques, Oleg Kiselyov, and Chung-chieh Shan. 2009. "Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages." *Journal of Functional Programming* 19 (5): 509–43. https://doi.org/10.1017/S0956796809007205.

Ekman, Torbjörn, and Gorel Hedin. 2007. "The JastAdd System — Modular Extensible Compiler Construction." *Science of Computer Programming* 69 (1–3): 14–26. https://doi.org/10.1016/j.scico.2007.02.003.

Emidio, Luis. 2026. "Staged Arrow Engines (SAE): Explicit Structure, Staging, and Interaction Nets for Stackless Spatial Execution."

Erdweg, Sebastian, Tijs van der Storm, Markus Völter, Laurence Tratt, Remi Bosman, William R. Cook, Arthur Gerritsen, et al. 2013. "Evaluating and Comparing Language Workbenches: Existing Approaches and Results." In *Software Language Engineering (SLE 2013)*, 8225:243–82. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-319-02654-1_1.

Gamma, Erich, and Kent Beck. 2003. *Contributing to Eclipse: Principles, Patterns, and Plug-Ins*. Addison-Wesley.

handl project. 2026a. "An n-Dimensional Arrow Calculus for Programmable Computation and Rewriting."

———. 2026b. “Host-Tensor: Host-Side Tensor Functions for Handl Runtimes.”
———. 2026c. “Kit-Prelude-Lang: The Canonical Eval-Path Language as YAML Data.”
———. 2026d. “The Handl Architecture Decision Record Series.”
———. 2026e. “The Handl Kernel: A Closed Arrow Calculus for Composable Languages.”
———. 2026f. “The Handl Platform Documentation.”
Hughes, John. 2000. “Generalising Monads to Arrows.” *Science of Computer Programming* 37 (1–3): 67–111. https://doi.org/10.1016/S0167-6423(99)00023-4.
Ierusalimschy, Roberto, Luiz Henrique de Figueiredo, and Waldemar Celes. 2007. “The Evolution of Lua.” In *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages (HOPL III)*, 2-1-2-26. ACM. https://doi.org/10.1145/1238844.1238846.
Kats, Lennart C. L., and Eelco Visser. 2010. “The Spoofax Language Workbench: Rules for Declarative Specification of Languages and IDEs.” In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 444–63. ACM. https://doi.org/10.1145/1869459.1869497.
Klint, Paul, Tijs van der Storm, and Jurgen Vinju. 2009. “RASCAL: A Domain Specific Language for Source Code Analysis and Manipulation.” In *Ninth IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 168–77. IEEE. https://doi.org/10.1109/SCAM.2009.28.
Lawvere, F. William. 1963. “Functorial Semantics of Algebraic Theories.” PhD thesis, Columbia University.
Lindley, Sam, Philip Wadler, and Jeremy Yallop. 2010. “The Arrow Calculus.” *Journal of Functional Programming* 20 (1): 51–77. https://doi.org/10.1017/S095679680999027X.
Microsoft. 2023. “Language Server Protocol Specification.” https://microsoft.github.io/language-server-protocol/.
Miller, Mark Samuel. 2006. “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.” PhD thesis, Johns Hopkins University.
mlopt project. 2026. “Graphopt: A Learned Rewrite Substrate on the Handl Platform.”
OSGi Alliance. 2020. “OSGi Core Release 8 Specification.” https://docs.osgi.org/specification/osgi.core/8.0.0/.
Sanada, Takahiro. 2024. “Algebraic Effects and Handlers for Arrows.” *Journal of Functional Programming* 34: e8. https://doi.org/10.1017/S0956796824000066.
Sloane, Anthony M. 2011. “Lightweight Language Processing in Kiama.” In *Generative and Transformational Techniques in Software Engineering III (GTTSE 2009), Revised Papers*, 6491:408–25. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-642-18023-1_12.
Van Wyk, Eric, Derek Bodin, Jimin Gao, and Lijesh Krishnan. 2010. “Silver: An Extensible Attribute Grammar System.” *Science of Computer Programming* 75 (1–2): 39–54. https://doi.org/10.1016/j.scico.2009.07.004.
Würthinger, Thomas, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. 2017. “Practical Partial Evaluation for High-Performance Dynamic Language Runtimes.” In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 662–76. ACM. https://doi.org/10.1145/3062341.3062381.
Würthinger, Thomas, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Greg Richards, Doug Simon, and Mario Wolczko. 2013. “One VM to Rule Them All.” In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*, 187–204. ACM. https://doi.org/10.1145/2509578.2509581.
Zenger, Matthias, and Martin Odersky. 2005. “Independently Extensible Solutions to the Expression

Problem." In *Proceedings of the 12th International Workshop on Foundations of Object-Oriented Languages (FOOL).*